Læringsportal for frontend-utviklere – Dokumentasjons (HTML, CSS og JavaScript)

Forfatter: Maung Ko
Fag: Informasjonsteknologi og medieproduksjon
Dato: 2026

1. Innledning

Målet med prosjektet var å lage en moderne, responsiv og tilgjengelig læringsportal for webutvikling. Løsningen består av flere sider (forside, HTML, CSS, JavaScript og Om), med en konsistent hero-seksjon, kort (cards) for innhold, og et kontaktskjema. Teknologiene som er brukt er HTML (struktur), CSS (utseende/layout) og JavaScript (interaktivitet).

2. HTML – Struktur og semantikk

2.1 Dokumentoppsett og språk

Alle sider starter med <!doctype html> og angir språk med lang="nb"/"no". I <head> defineres tegnsett (UTF-8), responsiv visning (<meta name="viewport">), tittel og SEO-metadata (beskrivelse og Open Graph for deling). Dette gir korrekt tolking, bedre søkbarhet og pen deling på sosiale medier.

```html
<!DOCTYPE html>
<html lang="no">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>IT utvikling Læringportalen</title>

  <!-- Basic SEO/Open Graph -->
  <meta name="description" content="IT utvikling Læringportalen. Lær grunnleggende ferdigheter innen frontend og mer." />
  <meta property="og:title" content="IT utvikling Læringportalen" />
  <meta property="og:description" content="IT utvikling Læringportalen." />
  <meta property="og:type" content="website" />
  <meta property="og:locale" content="nb_NO" />

  <link rel="stylesheet" href="../css/style.css" />
</head>
<body>
  <!-- Skip link for accessibility -->
  <a class="skip-link" href="#main">Hopp til hovedinnhold</a>

  <!-- Header / Nav -->
  <header class="site-header">
    <div class="container header-inner">
      <a class="brand" href="index.html" aria-label="Gå til forsiden">IT utvikling Læringportalen</a>

    <button class="nav-toggle" aria-expanded="false" aria-controls="site-nav">
        <span class="sr-only">Åpne/lukke meny</span> ☰
      </button>
```

2.2 Semantiske seksjoner

Jeg bruker semantiske HTML-elementer: <header> (toppfelt), <nav> (meny), <main id="main"> (hovedinnhold), <section> (tematiske avsnitt), <article> (cards) og <footer> (opphavsinfo). Dette gir tydelig struktur, bedre tilgjengelighet og SEO.

2.3 Hero-seksjonen

Heroen introduserer siden med en stor overskrift, kort beskrivelse og tre knapper (CTA). Bildet ligger i <figure class="hero-media">. Strukturen gir rask oversikt og tydelige valg.

```html
<!-- Hero -->
<section class="hero" role="region" aria-labelledby="hero-title">
  <div class="container hero-inner">
    <div class="hero-copy">
      <h1 id="hero-title">IT utvikling Læringportalen</h1>
      <p>Frontend grunnlag: HTML, CSS og JavaScript. Bygg moderne nettsider.</p>
      <div class="hero-cta">
        <a class="btn primary" href="html.html">Start med HTML</a>
        <a class="btn" href="css.html">Utforsk CSS</a>
        <a class="btn" href="java.html">JavaScript</a>
      </div>
    </div>
    <figure class="hero-media" aria-hidden="true">
      <img src="../images/utvikling.jpg" alt="utvikling" />
    </figure>
  </div>
</section>
```

2.4 Kursseksjon (cards)

Innhold presenteres i kort (<article class="card">) med bilde, tittel, tekst og knapp «Les mer». Strukturen er gjenbrukbar og enkel å utvide.

```html
<!-- Portalen oversikt -->
<main id="main">
  <section class="courses container" aria-labelledby="kurs-title">
    <h2 id="kurs-title">IT utvikling oversikt</h2>

    <div class="grid">
      <!-- HTML Card -->
      <article class="card">
        <img src="../images/html.png" alt="html" />
        <div class="card-body">
          <h3>HTML</h3>
          <p>HTML er grunnspråket for å lage nettsider. Det står for HyperText Markup Language, og brukes t
          <a class="btn small" href="html.html" aria-label="Les mer om HTML">Les mer</a>
        </div>
      </article>

      <!-- CSS Card -->
      <article class="card">
        <img src="../images/Css.png" alt="css" />
        <div class="card-body">
          <h3>CSS</h3>
          <p>CSS (Cascading Style Sheets) er språket som brukes for å bestemme utseendet på en nettside.
            Det styrer farger, størrelser, avstander, layout og mye mer.</p>
          <a class="btn small" href="css.html" aria-label="Les mer om CSS">Les mer</a>
        </div>
      </article>

      <!-- JavaScript Card -->
      <article class="card">
        <img src="../images/Javascript.gif" alt="js" />
        <div class="card-body">
          <h3>JavaScript</h3>
```

2.5 Tilgjengelighet

Skip-link til #main, aria-attributter som aria-current og aria-expanded gjør navigasjonen mer tilgjengelig for tastaturbrukere og skjermlesere.

3. CSS – Design, layout og responsivitet

style.css index.html Settings java.html

css > style.css > body

```
:root {
  --bg: #0f172a;
  --panel: #111827;
  --text: #e5e7eb;
  --muted: #9ca3af;
  --brand: #2563eb;
  --brand-2: #38bdf8;
  --card: #1f2937;
  --border: #374151;
}

/* Reset */
* { box-sizing: border-box; }
html, body { margin: 0; padding: 0; }
img { max-width: 100%; display: block; }
a { text-decoration: none; color: inherit; }
button, input, textarea { font: inherit; }

body {
  font-family:
  system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  color: var(--text);
  background: linear-gradient(180deg, var(--bg), #0b1225 40%, #0a0f1f);
  line-height: 1.6;
}

.container { width: min(1100px, 92vw); margin-inline: auto; }

/* Skip link */
.skip-link { position: absolute; left: -9999px; }
.skip-link:focus {
  left: 0; background: #fff; color: #000; padding: 0.5rem 1rem; z-index: 1000;
}

/* Header */
.site-header {
  position: sticky; top: 0; z-index: 50;
```

CSS-variabler i :root gir konsistent fargepalett. En liten reset fjerner standardmarginer og gjør bilder skalerbare. .container begrenser bredden (min(1100px, 92vw)) og sentrerer innhold. Hero bruker CSS Grid (1fr | clamp(280px, 35vw, 520px)). Cards legges i et grid med 3 kolonner som skalerer ned på mindre skjermer. Knapper og paneler har runde hjørner og skygge for dybde.

Bilder i hero har aspect-ratio: 4/3 og object-fit: cover for stabile proporsjoner uten forvrenging. Responsivitet håndteres med media queries ved 900px og 720px. Skip-link blir synlig ved fokus, og inputs får tydelig fokusramme.

4. JavaScript – Interaktivitet og automasjon

Mobilmenyen åpnes/lukkes ved å toggle .open på <nav> og oppdatere aria-expanded. «Les mer»-knapper styrer kun sin egen seksjon ved å vise/skjule .more-text og «…» (dots), og oppdaterer aria-expanded og knappetekst. I tillegg fyller koden automatisk inn årstall i footer via new Date().getFullYear().

```javascript


/* Read more / less (collapsed baseline) */
const buttons = document.querySelectorAll(".read-more-btn");
buttons.forEach((button) => {
  const card = button.closest(".card-body");
  const moreText = card?.querySelector(".more-text");
  const dots = card?.querySelector(".dots");

  if (moreText) moreText.classList.add("hidden");
  if (dots) dots.style.display = "inline";
  button.textContent = "Les mer";
  button.setAttribute("aria-expanded", "false");

  button.addEventListener("click", () => {
    const isExpanded = button.getAttribute("aria-expanded") === "true";
    button.setAttribute("aria-expanded", String(!isExpanded));
    if (moreText) moreText.classList.toggle("hidden", isExpanded);
    if (dots) dots.style.display = isExpanded ? "inline" : "none";
    button.textContent = isExpanded ? "Les mer" : "Vis mindre";
  });
});
```

5. Skjermbilder (figurer) med forklaringer

Figur 1 – Skjermbilde fra nettsiden

Forklaring: Illustrerer hero/kursseksjon slik den fremstår i nettleseren. Oppsettet følger CSS-grid og knappene bruker primær/sekundær stil.

6. Vurdering og faglige begrunnelser

Semantisk HTML gjør strukturen forståelig for både mennesker og maskiner. CSS-variabler og Grid/clamp() gir robust, gjenbrukbar og responsiv layout. JavaScript er minimalistisk og forbedrer UX uten å skape unødvendig kompleksitet. Tilgjengelighet ivaretas med skip-link, aria-attributter og fokusstiler.

7. Konklusjon

Nettsiden demonstrerer god praksis i frontend: strukturert HTML, moderne og responsiv CSS, samt enkel, tilgjengelig interaktivitet i JavaScript. Løsningen er modulær og kan utvides videre med nye sider og innhold.

8. Knytte arbeidet til kompetansemål på VG3-nivå

Arbeidet med å utvikle en responsiv og brukervennlig nettside dekker flere kompetansemål i læreplanen for IT-utviklerfaget (VG3).

8.1 Planlegging og dokumentasjon

Prosjektet viser planlegging, struktur og dokumentasjon i tråd med kompetansemål om å planlegge, gjennomføre og dokumentere utviklingsarbeid.

8.2 Bruk av programmeringsspråk

Jeg har brukt HTML, CSS og JavaScript for å utvikle løsningen, noe som viser kompetanse i flere programmeringsspråk og forståelse for struktur, design og interaktivitet.

8.3 Utvikling av brukergrensesnitt

Løsningen følger krav om brukervennlighet og universell utforming. Dette er i tråd med kompetansemål om å lage tilgjengelige og funksjonelle brukergrensesnitt.

8.4 Kvalitetssikring og testing

Testing av responsivitet, tilgjengelighet og funksjonalitet viser at jeg kan teste egen kode og vurdere kvalitet.

8.5 Sikkerhet og teknologisk forståelse

Selv uten sensitive data viser prosjektet forståelse for strukturert, trygg kode og god praksis.

9. Hva som kreves i lærefaget (IT-utvikler)

En lærling i IT-utviklerfaget må kunne:

- Forstå hele utviklingsprosessen

- Bruke programmeringsspråk og verktøy

- Jobbe både selvstendig og i team

- Tolke krav og levere løsninger som møter behov

10. Refleksjon over egen utvikling

Gjennom prosjektet har jeg utviklet meg innen HTML, CSS, JavaScript, universell utforming og dokumentasjon. Jeg har blitt mer selvstendig, bedre til å planlegge og mer bevisst på brukervennlighet og struktur. Jeg ønsker videre å forbedre meg innen rammeverk, testing og mer avanserte funksjoner.